EXPERIMENT 3

Write a Python Program for the Water Jug Problem

Aim

To develop a Python program to solve the **Water Jug Problem** using the **Breadth-First Search (BFS)** algorithm and determine the sequence of operations required to obtain the desired quantity of water.

Objective

- To understand state-space search problems in Artificial Intelligence.
- To apply the Breadth-First Search (BFS) algorithm.
- To determine the shortest sequence of operations required to obtain the target amount of water.
- To study problem-solving using state-space representation.

Theory

The **Water Jug Problem** is a classic Artificial Intelligence problem where two jugs of different capacities are used to measure a specific amount of water.

Suppose:

- Jug A = **4 Litres**
- Jug B = **3 Litres**
- Target = **2 Litres**

Allowed operations:

- Fill a jug completely.
- Empty a jug completely.
- Pour water from one jug to another until either the source jug becomes empty or the destination jug becomes full.

The objective is to obtain exactly **2 litres** of water.

Breadth-First Search (BFS) is commonly used because it guarantees the shortest sequence of operations.

Algorithm

Step 1: Start with both jugs empty (0,0).

Step 2: Insert the initial state into a queue.

Step 3: Mark the current state as visited.

Step 4: Generate all possible next states by:

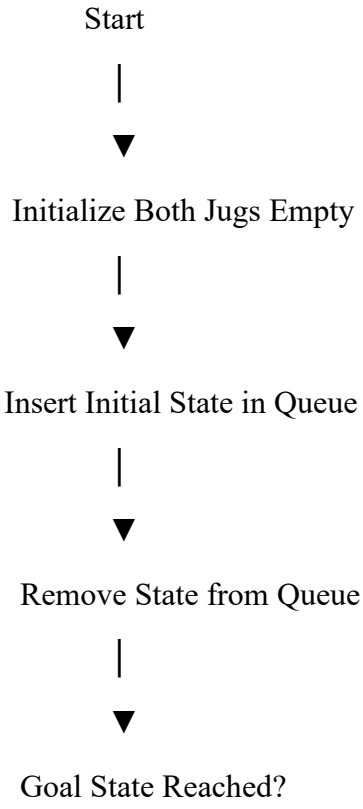
- Filling Jug A
- Filling Jug B
- Emptying Jug A
- Emptying Jug B
- Pouring Jug A → Jug B
- Pouring Jug B → Jug A

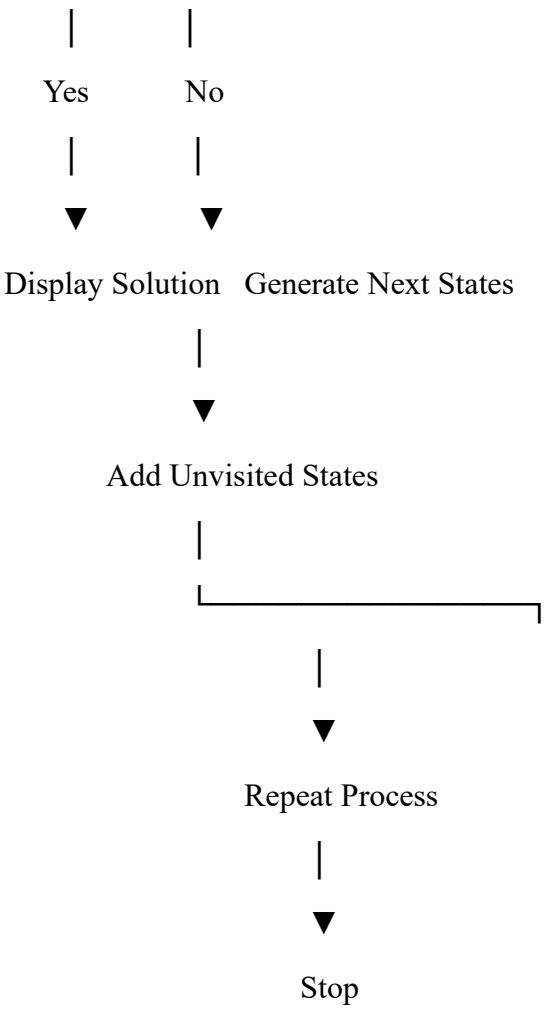
Step 5: If the target amount is reached, stop.

Step 6: Otherwise, insert unvisited states into the queue.

Step 7: Repeat until the goal state is found.

Flowchart





Python Program

```
from collections import deque

def water_jug_bfs(capacity1, capacity2, target):

    visited = set()
    queue = deque()

    queue.append((0, 0, []))

    while queue:
```

```
jug1, jug2, path = queue.popleft()

if (jug1, jug2) in visited:
    continue

visited.add((jug1, jug2))
path = path + [(jug1, jug2)]

if jug1 == target or jug2 == target:
    return path

next_states = [

    (capacity1, jug2),              # Fill Jug1
    (jug1, capacity2),              # Fill Jug2
    (0, jug2),                    # Empty Jug1
    (jug1, 0),                    # Empty Jug2

    (jug1 - min(jug1, capacity2-jug2),
     jug2 + min(jug1, capacity2-jug2)),  # Pour Jug1 -> Jug2

    (jug1 + min(jug2, capacity1-jug1),
     jug2 - min(jug2, capacity1-jug1))   # Pour Jug2 -> Jug1
]

for state in next_states:
    if state not in visited:
        queue.append((state[0], state[1], path))
```

```
    return None



capacity1 = 4
capacity2 = 3
target = 2

solution = water_jug_bfs(capacity1, capacity2, target)

if solution:
    print("Steps to reach the goal:\n")
    for step in solution:
        print(step)
else:
    print("No Solution Found")
```

Sample Output

Steps to reach the goal:

(0, 0)

(0, 3)

(3, 0)

(3, 3)

(4, 2)

Here, Jug B contains exactly **2 litres**, which is the required target.

Explanation of Output

State	Meaning
(0,0)	Both jugs are empty
(0,3)	Fill the 3-litre jug
(3,0)	Pour water from Jug B to Jug A
(3,3)	Fill Jug B again
(4,2)	Pour Jug B into Jug A until Jug A is full; Jug B now has 2 litres

Thus, the required quantity of **2 litres** is successfully obtained.

Advantages

- Guarantees the shortest sequence of operations.
- Explores all possible states systematically.
- Suitable for state-space search problems.
- Easy to implement using a queue.

Applications

- Artificial Intelligence search problems.
- Robotics and automated planning.
- Resource allocation.
- State-space problem solving.
- Puzzle-solving algorithms.

Result

The Python program successfully solved the **Water Jug Problem** using the **Breadth-First Search (BFS)** algorithm and determined the shortest sequence of operations required to obtain the desired quantity of water.

Conclusion

The **Water Jug Problem** demonstrates how state-space search techniques can be applied to solve real-world planning problems. By using the **Breadth-First Search (BFS)** algorithm, the program systematically explores all possible states and guarantees the shortest sequence

of operations to reach the target amount. This experiment highlights the practical application of BFS in Artificial Intelligence for solving constraint- based and search problems.

```
PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Var
/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC EXPs/EXP1.py"
Steps to reach the goal:

(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code>
```

EXP1.py

C: > Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.py > ...

```python
from collections import deque

def water_jug_bfs(capacity1, capacity2, target):

    visited = set()
    queue = deque()

    queue.append((0, 0, []))

    while queue:

        jug1, jug2, path = queue.popleft()

        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

        if jug1 == target or jug2 == target:
            return path

        next_states = [

            (capacity1, jug2),              # Fill Jug1
            (jug1, capacity2),              # Fill Jug2
            (0, jug2),                      # Empty Jug1
            (jug1, 0),                      # Empty Jug2

            (jug1 - min(jug1, capacity2-jug2),
             jug2 + min(jug1, capacity2-jug2)),  # Pour Jug1 -> Jug2

            (jug1 + min(jug2, capacity1-jug1),
             jug2 - min(jug2, capacity1-jug1))   # Pour Jug2 -> Jug1
        ]

        for state in next_states:
            if state not in visited:
                queue.append((state[0], state[1], path))

    return None


capacity1 = 4
capacity2 = 3
target = 2

solution = water_jug_bfs(capacity1, capacity2, target)

if solution:
    print("Steps to reach the goal:\n")
    for step in solution:
        print(step)
else:
    print("No Solution Found")
```